Multi-Label Classification

A **multi-label classification** problem is one where a single input, X, can be associated with **multiple output labels** at the same time.

This is different from multi-class classification, where each input has only *one* label, even if that label can be one of many possible values (e.g., $y = 7$).

Example: Autonomous Driving 🚗

A common example is scene detection for a self-driving car.

- **Input (X):** An image from the car's camera.
- **Task:** Identify all objects present in the image.
- **Output Labels (y):** The output is a **vector** of N binary labels, one for each possible object.
    - Is there a car? (Yes/No)
    - Is there a bus? (Yes/No)
    - Is there a pedestrian? (Yes/No)

A single image could have an output vector of `y = [1, 0, 1]`, meaning "Car" and "Pedestrian" are present, but "Bus" is not.

Multi-Label vs. Multi-Class

This table summarizes the key difference:

Feature	Multi-Class Classification	Multi-Label Classification
Example	Handwritten Digit (0-9)	Scene Detection (Car, Bus, Pedestrian)
Output `y`	A single number (e.g., `y = 7`)	A vector of numbers (e.g., `y = [1, 0, 1]`)
Question	"Which *one* of these classes is this?"	"Which *of these classes* (zero or more) are present?"
Output Layer	**Softmax** (forces probabilities to sum to 1)	**Sigmoid** (applied to each neuron independently)

Neural Network Implementation

You can build a single neural network to solve a multi-label problem.

- **Hidden Layers:** These are standard (e.g., using `relu` activations).
- **Output Layer:** This is the key difference.
    - The output layer must have **N neurons**, where N is the number of labels you are trying to predict (e.g., 3 neurons for car/bus/pedestrian).
    - The activation function for *each* of these neurons must be **sigmoid**.

This effectively treats the problem as N separate binary classification problems that the network learns to solve simultaneously.